

CSCI3150 Midterm Examination

Exam Information:

- Course: **CSCI3150 Intro to Operating Systems, Spring 2026**
- Time Limit: **25 mins**
- Total Questions: **4**
- Total Pages: **5**

Please put your FULL NAME and Student ID on THIS page only.

- Name:
- Student ID:

Questions	Total Marks	Marks Received
Question 1	1	
Question 2	1	
Question 3	2	
Question 4	1	
Total	5	

Please Read All Questions Carefully. Gook Luck!

Q1 (1 mark)

Multiple choices. There are one or more correct choices for each question. Mark all of them. For each question, you will get 0.5 marks only if your answer is 100% correct, otherwise you will get 0.

Question 1.a (0.5 marks) (Multiple choices) Which of the following statements is/are correct about system call and processes?

- A. When a user process makes a system call, it requires the CPU to switch from user mode to kernel mode.
- B. After calling `exec()`, a process can still access a variable that is stored in the stack before calling `exec()`.
- C. After a successful `fork()` system call, the newly created child process will start executing from the very first instruction of the `main()` function.
- D. If the standard output of a process is redirected to a file using `dup2()`, then this process can not print anything to standard output in the future.

Answer: A

Question 1.b (0.5 marks) (Multiple choices) Two threads in the same process do not share:

- A. Heap
- B. File descriptors
- C. Stack pointer
- D. Global variables

Answer: C

Q2 (1 mark)

Question 2. Read the following program and answer the questions:

```

#include <stdio.h>
#include <unistd.h>
void main(){
    int i;
    int pid;
    for(i=0;i<4;i++)
    {
        if(i % 2 == 0){
            pid = fork();
            if(pid == 0){
                printf("#\n");
            }
        }
        else{
            printf("!\n");
        }
    }
    return;
}

```

- How many processes are created when we execute this program once in a Shell, including the first process created by the shell. No explanation is needed.
- How many times would # and ! be output?

Answer: a) 4 b) 3 for "#", 6 for "!"

Q3. (2 marks)

Question 3.

Answer: (4x0.3 + 2x0.4 marks)

- A
- E
- D
- F
- G
- H

Note: The partial mark for this question is based on the number of TODOs you answer correctly:

# correct TODOs	0	1	2	3	4	5	6
Marks received	0	0.3	0.6	0.9	1.2	1.6	2

```

#define MAX_MEAT 5
#define MAX_BREAD 10

```

```

pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cond_meat = PTHREAD_COND_INITIALIZER;
pthread_cond_t cond_bread = PTHREAD_COND_INITIALIZER;

```

```

int meat = 0;
int bread = 0;
int shawarma = 0;

```

```

void* meat_supplier(void* arg) {
    while (1) {
        pthread_mutex_lock(&mutex);
        if (/*TODO 1*/) {
            meat = MAX_MEAT;
            printf("Meat supplier refilled the meat.\n");
            /*TODO 2*/
        }
        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}

void* bread_supplier(void* arg) {
    while (1) {
        pthread_mutex_lock(&mutex);
        if (/*TODO 3*/) {
            bread++;
            printf("Bread supplier added a bread. Total bread: %d\n", bread);
            /*TODO 4*/
        }
        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}

void* producer(void* arg) {
    while (1) {
        pthread_mutex_lock(&mutex);
        while (meat < 1) {
            /*TODO 5*/
        }
        while (bread < 2) {
            /*TODO 6*/
        }
        // Produce one shawarma
        meat--;
        bread -= 2;
        printf("Producer made a shawarma.\n");
        shawarma++;
        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}

```

Answer:

Q4 (1 mark)

Question 4: In your shawarma restaurant, customers must have a seat and then receive service from a waiter before they can begin ordering. When the order is taken, the waiter leaves. When the meal is finished, the customer leaves the seat. This process can be simulated in C syntax (All the necessary header files have already been included).

```

#define NUM_SEATS 6      // Number of available seats
#define NUM_WAITERS 4    // Number of available waiters
#define NUM_CUSTOMERS 12 // Number of customers

sem_t seats;    // Semaphore for available seats
sem_t waiters;  // Semaphore for available waiters

void* customer(void* type) {
    /*TODO 1*/
    /*TODO 2*/

    order(); // The customer is ordering.
    /*TODO 3*/
    eat(); // The customer finishes eating.
    /*TODO 4*/
    pthread_exit(NULL);
}

int main() {
    pthread_t customers[NUM_CUSTOMERS];

    sem_init(&seats, 0, NUM_SEATS);
    sem_init(&waiters, 0, NUM_WAITERS);

    for (int i = 0; i < NUM_CUSTOMERS; i++) {
        pthread_create(&customers[i], NULL, customer, &customer_types[i]);
    }
    /* ... */
}

```

You are required to fill in the TODOs in the code. Each TODO only need one line of code.

Hint – semaphore related POSIX APIs that may be used:

```
int sem_wait(sem_t *);
```

```
int sem_post(sem_t *);
```

Answer: (0.25 marks each) 1. `sem_wait(&seats);` 2. `sem_wait(&waiters);` 3. `sem_post(&waiters);` 4. `sem_post(&seats);`

END